**CSE225L – Data Structures and Algorithms Lab**
**Lab 06**
**Unsorted List (array based)**

In today's lab we will design and implement the List ADT where the items in the list are unsorted.

**unsortedtype.h**

```
#ifndef UNSORTEDTYPE_H_INCLUDED
#define UNSORTEDTYPE_H_INCLUDED

const int MAX_ITEMS = 5;

template <class ItemType>
class UnsortedType
{
    public :
        UnsortedType();
        void MakeEmpty();
        bool IsFull();
        int LengthIs();
        void InsertItem(ItemType);
        void DeleteItem(ItemType);
        void RetrieveItem(ItemType&, bool&);
        void ResetList();
        void GetNextItem(ItemType&);
    private:
        int length;
        ItemType info[MAX_ITEMS];
        int currentPos;
};
#endif // UNSORTEDTYPE_H_INCLUDED
```

**unsortedtype.cpp**

```
#include "UnsortedType.h"

template <class ItemType>
UnsortedType<ItemType>::UnsortedType()
{
    length = 0;
    currentPos = -1;
}
template <class ItemType>
void UnsortedType<ItemType>::MakeEmpty()
{
       length = 0;
}
template <class ItemType>
bool UnsortedType<ItemType>::IsFull()
{
    return (length == MAX_ITEMS);
}
template <class ItemType>
int UnsortedType<ItemType>::LengthIs()
{
    return length;
}
template <class ItemType>
void UnsortedType<ItemType>::ResetList()
{
    currentPos = -1;
}
template <class ItemType>
void
UnsortedType<ItemType>::GetNextItem(ItemType&
item)
{
    currentPos++;
    item = info [currentPos] ;
}
template <class ItemType>
void
UnsortedType<ItemType>::RetrieveItem(ItemType&
item, bool &found)
{
    int location = 0;
    bool moreToSearch = (location < length);
    found = false;
    while (moreToSearch && !found)
    {
        if(item == info[location])
        {
            found = true;
            item = info[location];
        }
        else
        {
            location++;
            moreToSearch = (location < length);
        }
    }
}
template <class ItemType>
void UnsortedType<ItemType>::InsertItem(ItemType
item)
{
    info[length] = item;
    length++;
}
template <class ItemType>
void UnsortedType<ItemType>::DeleteItem(ItemType
item)
{
    int location = 0;
    while (item != info[location])
        location++;
    info[location] = info[length - 1];
    length--;
}
```

Generate the **driver file (main.cpp)** where you perform the following tasks. Note that you cannot make any change to the header file or the source file.

| **Operation to Be Tested and Description of Action** | **Input Values** | **Expected Output** |
|---|---|---|
| • Create a list of integers | | |
| • Insert four items | 5 7 6 9 | |
| • Print the list | | 5 7 6 9 |
| • Print the length of the list | | 4 |
| • Insert one item | 1 | |
| • Print the list | | 5 7 6 9 1 |
| • Retrieve 4 and print whether found or not | | Item is not found |
| • Retrieve 5 and print whether found or not | | Item is found |
| • Retrieve 9 and print whether found or not | | Item is found |
| • Retrieve 10 and print whether found or not | | Item is not found |
| • Print if the list is full or not | | List is full |
| • Delete 5 | | |
| • Print if the list is full or not | | List is not full |
| • Delete 1 | | |
| • Print the list | | 7 6 9 |
| • Delete 6 | | |
| • Print the list | | 7 9 |
| • Write a class `studentInfo` that represents a student record. It must have variables to store the student ID, student's name and student's CGPA. It also must have a function to print all the values. You will also need to overload a few operators. | | |
| • Create a list of objects of class `studentInfo`. | | |
| • Insert 5 student records | 15234 Jon 2.6<br>13732 Tyrion 3.9<br>13569 Sandor 1.2<br>15467 Ramsey 2 3.1<br>16285 Arya 3.1 | |
| • Delete the record with ID 15467 | | |
| • Retrieve the record with ID 13569 and print whether found or not along with the entire record | | Item is found<br>13569, Sandor, 1.2 |
| • Print the list | | 15234, Jon, 2..6<br>13732, Tyrion, 3.9<br>13569, Sandor, 1.2<br>16285, Arya, 3.1 |

**StudentInfo.h**

```
#ifndef STUDENTINFO_H_INCLUDED
#define STUDENTINFO_H_INCLUDED

#include <iostream>
#include <string>
using namespace std;

class StudentInfo {
private:
   int studentID;
   string name;
   float cgpa;

public:
   StudentInfo();
   StudentInfo(int id, string n, float g);
   void printRecord();

   // Overload operators
   bool operator==(StudentInfo other);
   bool operator!=(StudentInfo other);

   int getStudentID();  // Getter
};

#endif // STUDENTINFO_H_INCLUDED
```

**StudentInfo.cpp**

```
#include "StudentInfo.h"

StudentInfo::StudentInfo() {
   studentID = 0;
   name = "";
   cgpa = 0.0;
}

StudentInfo::StudentInfo(int id, string n, float g) {
   studentID = id;
   name = n;
   cgpa = g;
}

void StudentInfo::printRecord() {
   cout << studentID << ", " << name << ", " << cgpa << endl;
}

// Overload the equality operator (==)
bool StudentInfo::operator==(StudentInfo other) {
   return (studentID == other.studentID);
}

// Overload the inequality operator (!=)
bool StudentInfo::operator!=(StudentInfo other) {
   return (studentID != other.studentID);
}

// Getter for studentID
int StudentInfo::getStudentID() {
   return studentID;
}
```

**Inlude the template datatypes explicitly at the end of unsortedtype.cpp. Also include the StudentInfo.h header file in your UnsortedType.cpp source file.**

```
template class UnsortedType<int>;
template class UnsortedType<StudentInfo>;
```

```cpp
#include <iostream>
#include "UnsortedType.h" // Include the header file for UnsortedType
#include "StudentInfo.h"

using namespace std;

int main()
{
     UnsortedType<int> intList;

    // Insert four items: 5, 7, 6, 9
    intList.InsertItem(5);
    intList.InsertItem(7);
    intList.InsertItem(6);
    intList.InsertItem(9);

    // Print the list: 5, 7, 6, 9
    cout << "List of integers: ";
    intList.ResetList();
    int item;
    for (int i = 0; i < intList.LengthIs(); i++) {
        intList.GetNextItem(item);
        cout << item << " ";
    }
    cout << endl;

    // Print the length of the list: 4
    cout << "Length of the list: " << intList.LengthIs() << endl;

    // Insert one item: 1
    intList.InsertItem(1);

    // Print the updated list: 5, 7, 6, 9, 1
    cout << "Updated list of integers: ";
    intList.ResetList();
    for (int i = 0; i < intList.LengthIs(); i++) {
        intList.GetNextItem(item);
        cout << item << " ";
    }
    cout << endl;

    // Retrieve 4 and print whether found or not
    int searchItem = 4;
    bool found = false;
    intList.RetrieveItem(searchItem, found);
    if (found) {
        cout << "Item 4 is found." << endl;
    } else {
        cout << "Item 4 is not found." << endl;
    }

    // Retrieve 5 and print whether found or not
    searchItem = 5;
    intList.RetrieveItem(searchItem, found);
    if (found) {
        cout << "Item 5 is found." << endl;
    } else {
        cout << "Item 5 is not found." << endl;
    }

    // Retrieve 9 and print whether found or not
    searchItem = 9;
    intList.RetrieveItem(searchItem, found);
    if (found) {
        cout << "Item 9 is found." << endl;
    } else {
```

```
        cout << "Item 9 is not found." << endl;
    }

    // Retrieve 10 and print whether found or not
    searchItem = 10;
    intList.RetrieveItem(searchItem, found);
    if (found) {
        cout << "Item 10 is found." << endl;
    } else {
        cout << "Item 10 is not found." << endl;
    }

    // Print if the list is full or not
    if (intList.IsFull()) {
        cout << "List is full." << endl;
    } else {
        cout << "List is not full." << endl;
    }

    // Delete 5
    intList.DeleteItem(5);

    // Print if the list is full or not
    if (intList.IsFull()) {
        cout << "List is full." << endl;
    } else {
        cout << "List is not full." << endl;
    }

    // Delete 1
    intList.DeleteItem(1);

    // Print the updated list: 7, 6, 9
    cout << "Updated list of integers after deletions: ";
    intList.ResetList();
    for (int i = 0; i < intList.LengthIs(); i++) {
        intList.GetNextItem(item);
        cout << item << " ";
    }
    cout << endl;

    // Delete 6
    intList.DeleteItem(6);

    // Print the updated list: 7, 9
    cout << "Updated list of integers after deleting 6: ";
    intList.ResetList();
    for (int i = 0; i < intList.LengthIs(); i++) {
        intList.GetNextItem(item);
        cout << item << " ";
    }
    cout << endl;

    return 0;
}
```

```cpp
#include <iostream>
#include "UnsortedType.h" // Include the header file for UnsortedType
#include "StudentInfo.h"

using namespace std;

int main()
{
    UnsortedType<StudentInfo> studentList;

    StudentInfo obj1 (15234, "Jon", 2.6);
    StudentInfo obj2 (13732, "Tyrion", 3.9);
    StudentInfo obj3 (13569, "Sandor", 1.2);
    StudentInfo obj4 (15467, "Ramsey", 3.1);
    StudentInfo obj5 (16285, "Arya", 3.1);
    // Inserting student records
    studentList.InsertItem(obj1);
    studentList.InsertItem(obj2);
    studentList.InsertItem(obj3);
    studentList.InsertItem(obj4);
    studentList.InsertItem(obj5);

    //Delete the record with ID 15467
    studentList.DeleteItem(StudentInfo(15467, "", 0));

    //Retrieve the record with ID 13569 and print whether found or not
    StudentInfo searchStudent(13569, "", 0);
    bool found1 = false;
    studentList.RetrieveItem(searchStudent, found);
    if (found1)
    {
        cout << "Item is found: ";
        searchStudent.printRecord();
    }
    else
    {
        cout << "Item is not found" << endl;
    }

    // Print the list of student records
    cout << "Student List: " << endl;
    studentList.ResetList();  // Reset the list to start from the beginning
    StudentInfo studentitem;
    for (int i = 0; i < studentList.LengthIs(); i++)
    {
        studentList.GetNextItem(studentitem);
        studentitem.printRecord();
    }

    return 0;
}
```